

DECIDABLE AND UNDECIDABLE PROBLEMS

Dr. A. Beulah
AP/CSE

LEARNING OBJECTIVE

- To Design Turing machines for any Languages (K3)
 - To Understand the concept of Decidable and Undecidable Problems

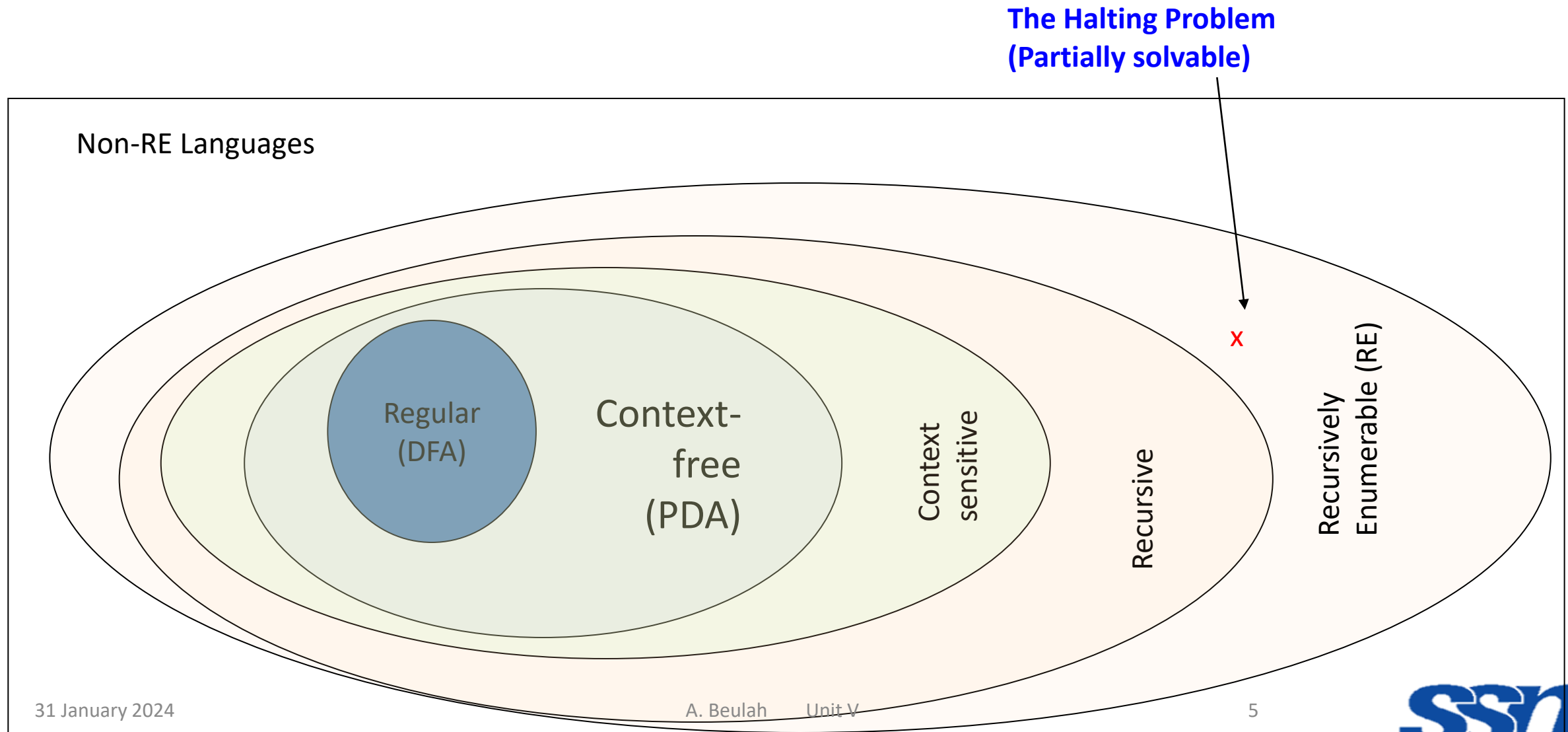
DECIDABLE PROBLEMS

- **Decidable problems about regular Languages**
 - Acceptance problem for DFAs
 - Acceptance problem for NFAs
 - Acceptance problem for Regular Expressions
 - Emptiness testing for DFAs
 - 2 DFAs recognizing the same language
- **Decidable problems about Context Free Languages**
 - Does a given CFG generate a given string?
 - Is the language of a given CFG empty?
 - Every CFL is decidable by a Turing machine

UNDECIDABLE PROBLEMS

- Halting Problem
- Post's Correspondence problem
- Busy Beaver problem
- Whether the language accepted by a TM is empty
- Whether the language accepted by a TM is regular language
- Whether the language accepted by a TM is context free language

UNDECIDABLE PROBLEMS

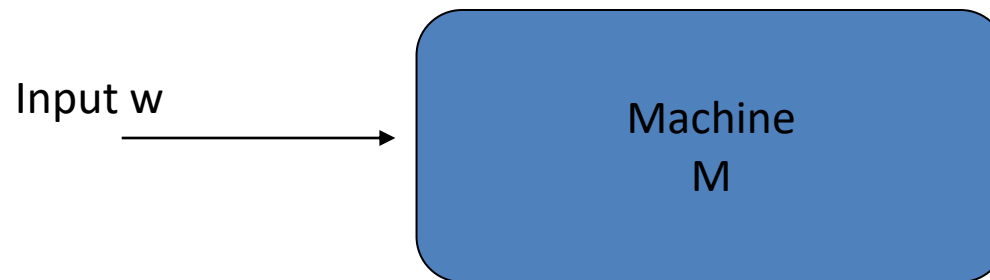


THE HALTING PROBLEM

- An example of a recursive enumerable problem that is also undecidable

WHAT IS THE HALTING PROBLEM?

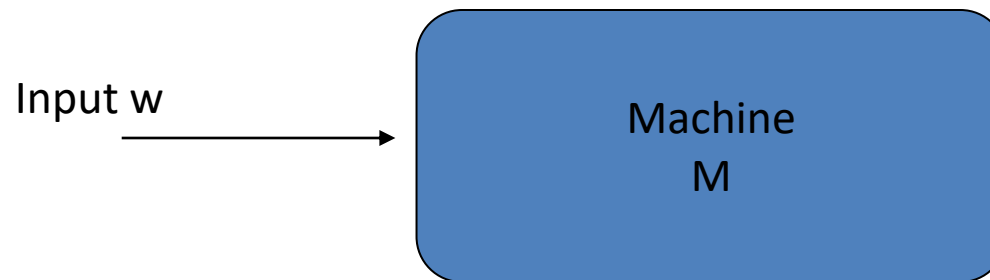
- Does a given Turing Machine M halt on a given input w ?
- or
- Is it possible to tell whether a given machine will halt for some given input?



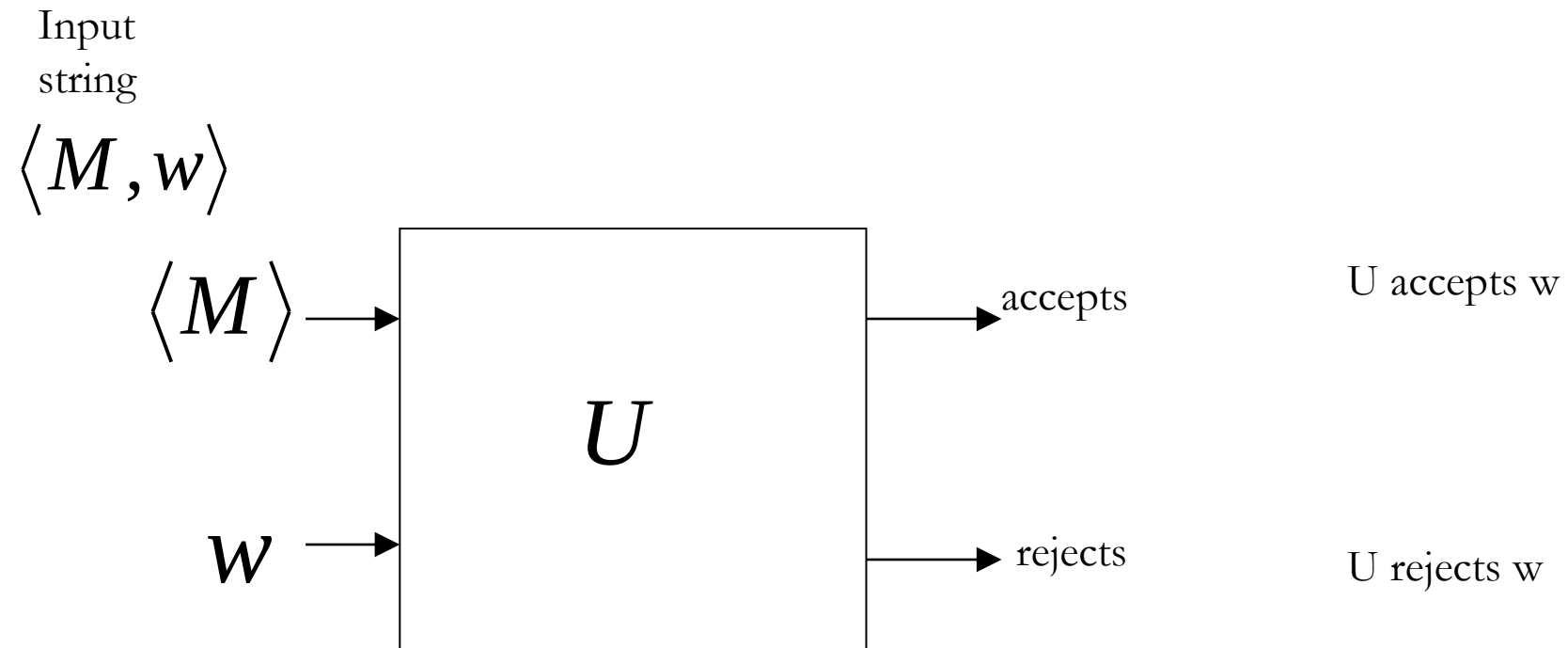
WHAT IS THE HALTING PROBLEM?

- Example: Given an arbitrary Turing machine M over alphabet $\Sigma = \{a, b\}$, and an arbitrary string w over Σ , does M halt when it is given w as an input ?

$$\text{HALT}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM which halts on } w \}$$



REVISIT UTM

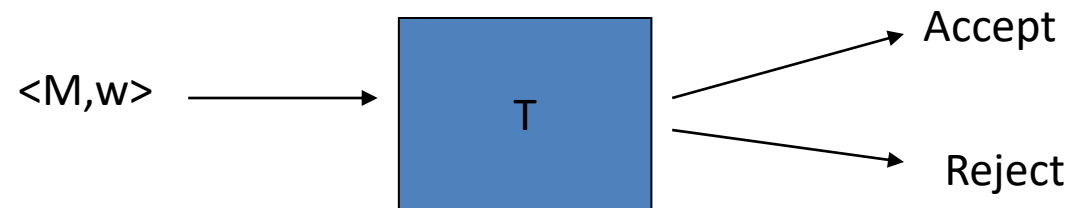


THEOREM: IS HALT_{TM} DECIDABLE?

- Halting Problem is undecidable
- If there was such a Turing Machine
 - Its input will have two portions, M and w
 - It outputs either a YES or a NO depending on whether M halts on input w

PROOF BY CONTRADICTION

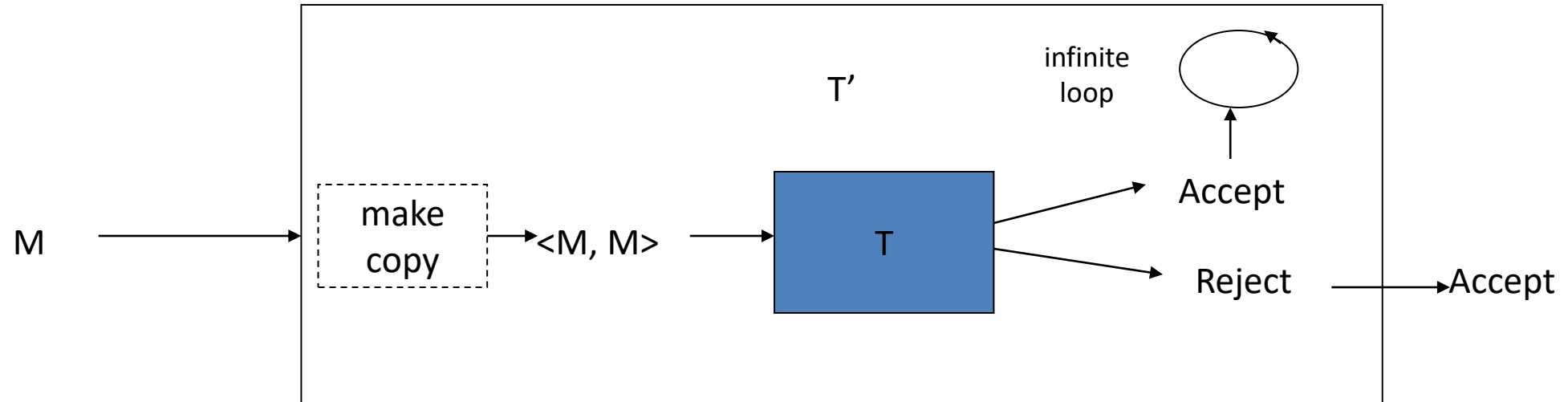
- Suppose Halting Problem is decidable
 - Plan: arrive at a contradiction
- If Halting Problem is decidable,
 - then there exists a TM T that decides Halting Problem



PROOF BY CONTRADICTION

- Create a TM T' based on T as follows:
 - T takes in a TM M
 - In T' , M is duplicated so that there are now two portions on the input tape
 - Feed this new input into T
 - When it is about to print reject, print accept instead
 - When it is about to print accept, send the program to an infinite loop

PROOF BY CONTRADICTION

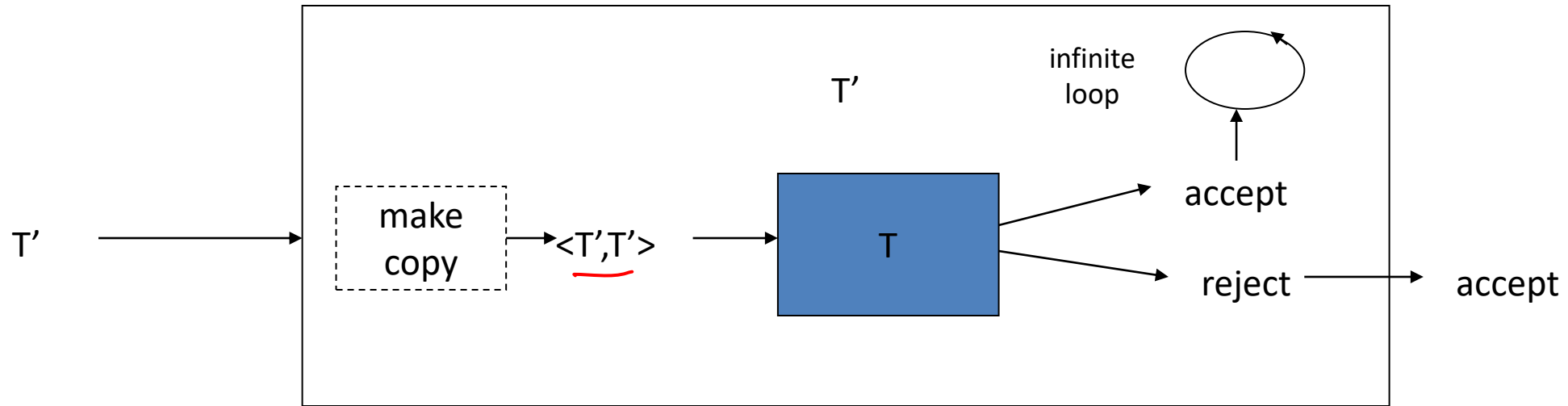


- Program T' takes a M as input, prints accept if M **does not halt** on input M , but goes into an infinite loop if M **halts** on input M

PROOF BY CONTRADICTION

- Consider feeding TM T' to itself
- Consequence (two possibilities)
 - It prints accept
 - T' halts on input T'
if T' does not halt on input $T' \rightarrow$ a contradiction
 - It goes to an infinite loop
 - T' does not halt on input T'
if T' halts on input $T' \rightarrow$ a contradiction
- Therefore the supposition cannot hold, and Halting Problem is undecidable

PROOF BY CONTRADICTION

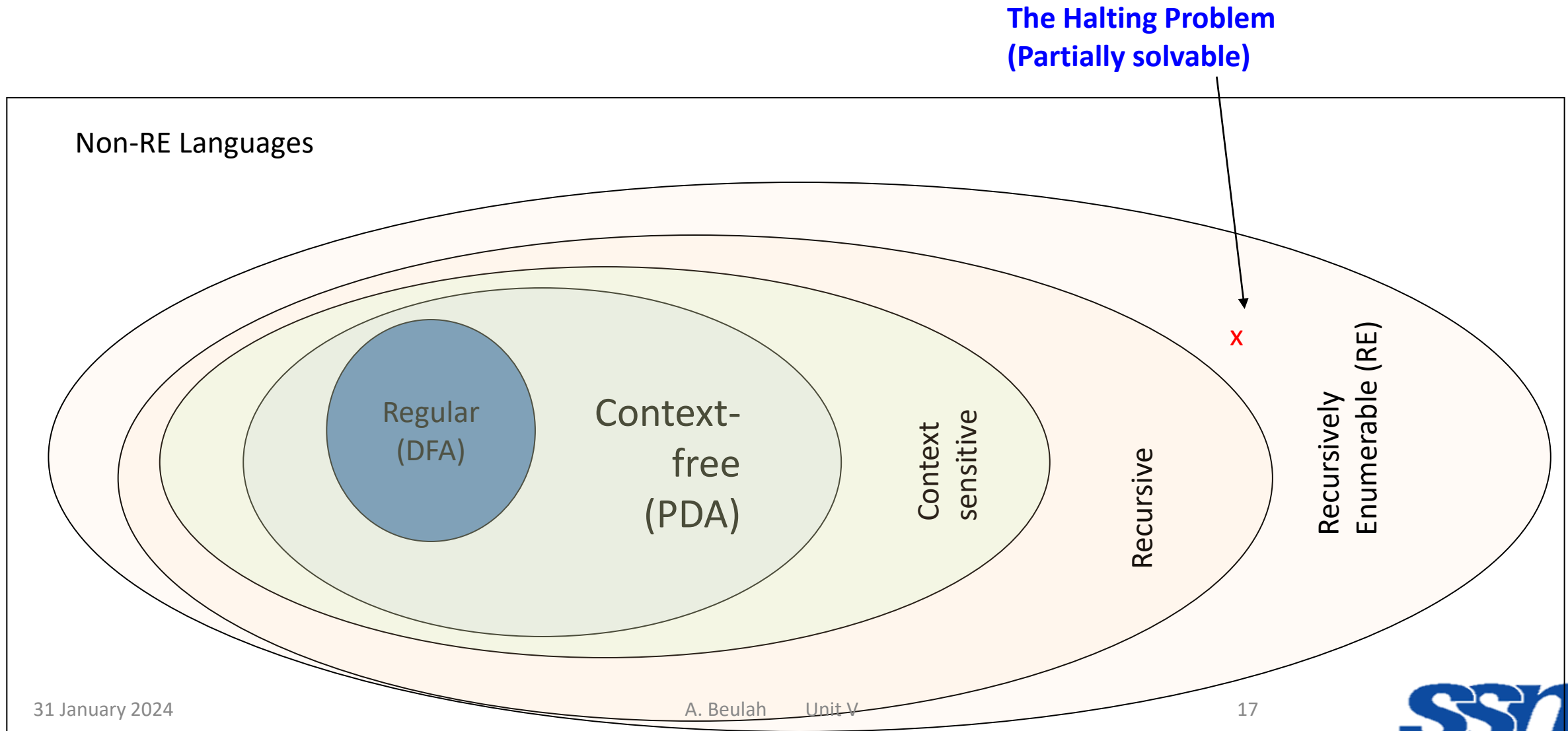


- T' halts on input T' (prints a accept, see outer box) if
- T' does not halt on input T' (T should yield a reject, see inner box)
- T' does not halt on input T' (infinite loop, see outer box) if
- T' halts on input T' (T should yield a accept, see inner box)

HP IS SEMIDECIDABLE

- There are problems such as HP that cannot be solved
- Actually, HP is semidecidable, that is if all we need is print accept when M on w halts, but not worry about printing reject if otherwise, a TM machine exists for the halting problem
 - Just simulate M on w , print accept (or go to a final state) when the simulation stops
 - This means that HP is not recursive but it is recursively enumerable

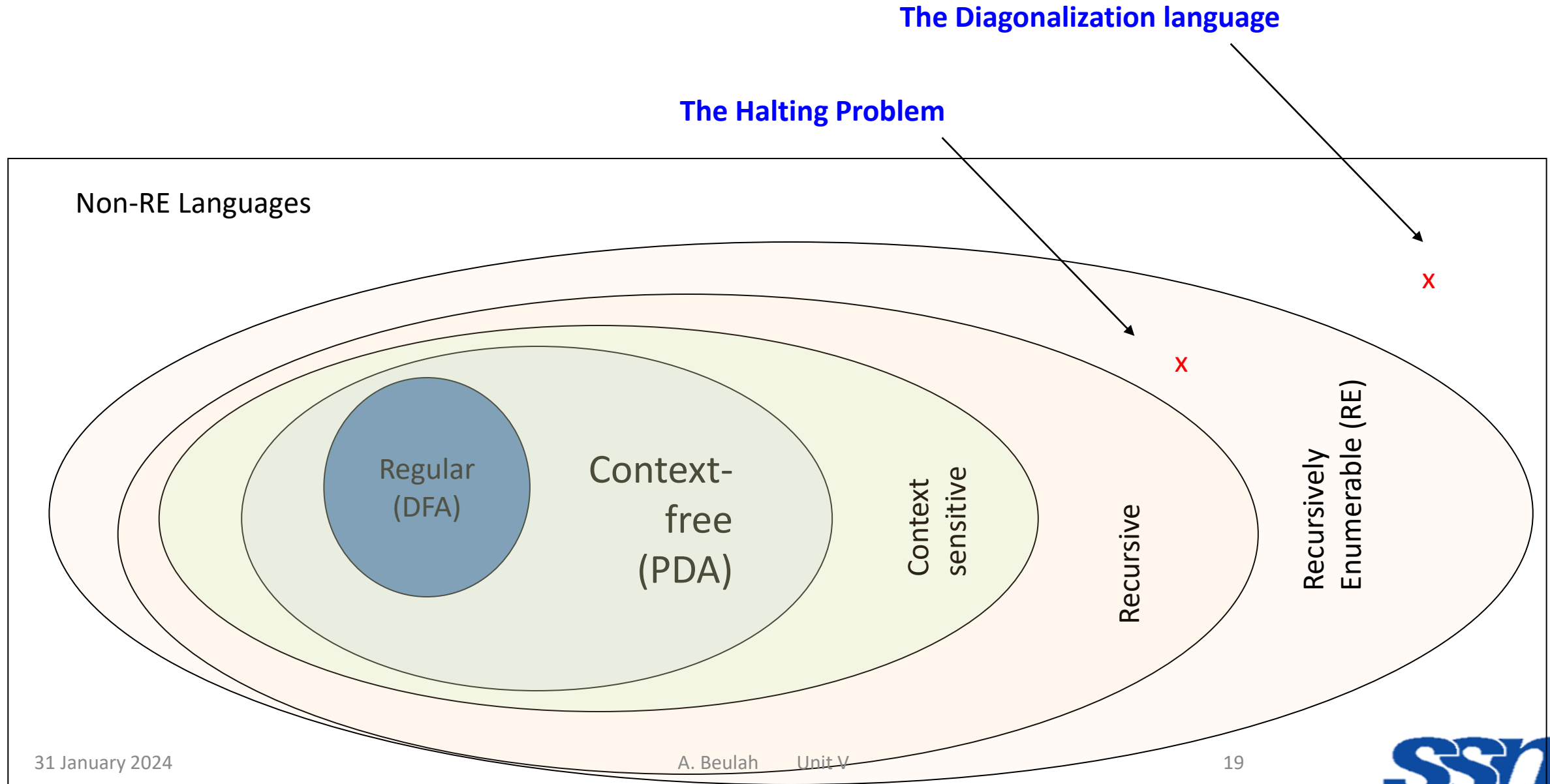
HP IS SEMIDECIDABLE



THE DIAGONALIZATION LANGUAGE

- Example of a language that is not recursive enumerable
- (i.e, no TMs exist)

THE DIAGONALIZATION LANGUAGE



A LANGUAGE ABOUT TMS & ACCEPTANCE

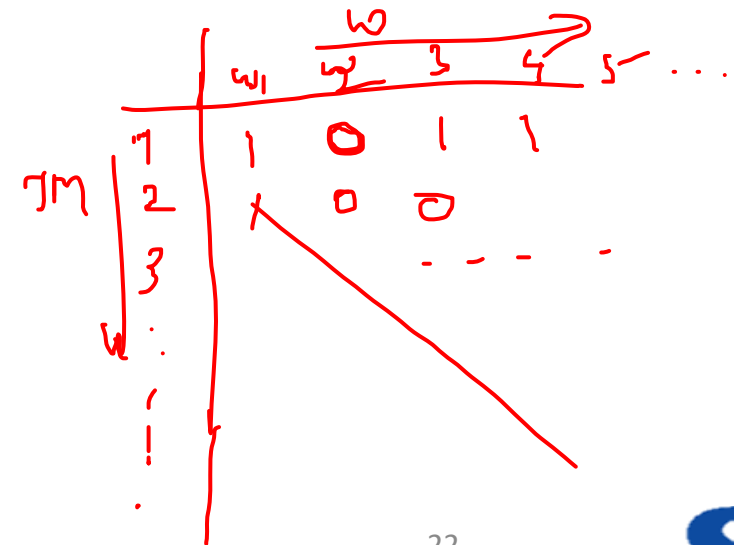
- Let L be the language of all strings $\langle M, w \rangle$ s.t.:
 1. M is a TM (coded in binary) with input alphabet also binary
 2. w is a binary string
 3. M accepts input w .

ANY TM M IN BINARY-CODED FORM

- $M = \{ Q, \{0,1\}, \Gamma, \delta, q_0, B, F \}$
 - Map all states, tape symbols and transitions to integers
($\rightarrow$ binary strings)
 - $\delta(q_i, X_j) = (q_k, X_l, D_m)$ will be represented as:
 $\rightarrow 0^i 1 0^j 1 0^k 1 0^l 1 0^m$
- Result: Each TM can be written down as a long binary string
- Canonical ordering of TMs:
 - $\{M_1, M_2, M_3, M_4, \dots, M_i, \dots\}$

ENUMERATING ALL BINARY STRINGS

- Let w be a binary string
- Then $1w \equiv i$, where i is some integer
 - E.g., If $w=\epsilon$, then $i=1$;
 - If $w=0$, then $i=2$;
 - If $w=1$, then $i=3$; so on...
- If $1w \equiv i$, then call w as the i^{th} word or i^{th} binary string, denoted by w_i .
- **A canonical ordering of all binary strings:**
 - $\{\epsilon, 0, 1, 00, 01, 10, 11, 000, 100, 101, 110, \dots\}$
 - $\{w_1, w_2, w_3, w_4, \dots, w_i, \dots\}$



THE DIAGONALIZATION LANGUAGE

- $L_d = \{ \underline{w_i} \mid \underline{w_i} \notin \underline{L(M_i)} \}$

- The language of all strings whose corresponding machine does *not* accept itself (i.e., its own code)

(TMs) i ↓

		j (input word w)			
		1	2	3	4
1	0	1	0	1	...
2	1	1	0	0	...
3	0	1	0	1	...
4	1	0	0	1	...

⋮

diagonal

- Table: $T[i,j] = 1$, if M_i accepts w_j
 $= 0$, otherwise.

- Make a new language called

$$\underline{L_d = \{w_i \mid T[i,i] = 0\}}$$

L_D IS NOT RE (I.E., HAS NO TM)

Proof (by contradiction):

Let M be the TM for L_D

→ M has to be equal to some M_k s.t.

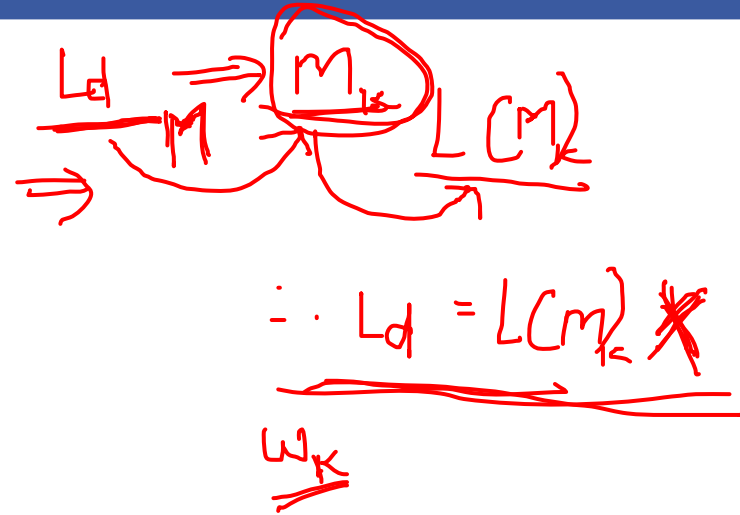
$$L(M_k) = L_D$$

→ Will w_k belong to $L(M_k)$ or not?

1. If $w_k \in L(M_k) \implies T[k,k]=1 \implies \underline{w_k \notin L_D}$

2. If $w_k \notin L(M_k) \implies \underline{T[k,k]=0} \implies \underline{\underline{w_k \in L_D}}$

A contradiction either way!!



POST'S CORRESPONDENCE PROBLEM

- Emil Post

DEFINITION

Given two lists A and B:

$$A = \underline{w_1}, w_2, \dots, w_k \quad B = \underline{x_1}, x_2, \dots, x_k$$

The problem is to determine if there is a sequence of one or more integers $i_1, i_2, \dots, i_m$ such that:

$$\underline{w_{i_1} w_{i_2} \dots w_{i_m}} = \underline{x_{i_1} x_{i_2} \dots x_{i_m}}$$

(w_i, x_i) is called a corresponding pair.

Indices may be repeated or omitted

EXAMPLE

$$A: \quad \begin{array}{ccc} \underline{w_1} & \underline{w_2} & \underline{w_3} \\ 100 & 11 & 111 \end{array}$$

$$B: \quad \begin{array}{ccc} x_1 & \underline{x_2} & x_3 \\ 001 & 111 & 11 \end{array}$$

PC-solution:

2,1,3

$$\underline{w_2 w_1 w_3} = x_2 x_1 x_3$$

$$\begin{array}{c} \text{Handwritten: } (w_2, w_1, w_3, x_2, x_1, x_3) \\ \underline{2, 1, 3} \\ \begin{array}{ccc} w_2 & w_1 & w_3 \\ 11 & 100 & 111 \end{array} = \begin{array}{ccc} x_2 & x_1 & x_3 \\ 111 & 001 & 11 \end{array} \end{array}$$

11100111

EXAMPLE

	w_1	w_2	w_3
$A :$	00	001	1000
	x_1	x_2	x_3
$B :$	0	11	011

- There is no solution
- Because total length of strings from B is smaller than total length of strings from A

MODIFIED POST CORRESPONDENCE PROBLEM (MPCP)

Given two lists A and B:

$$A = \underline{w_1}, w_2, \dots, w_k \quad B = x_1, x_2, \dots, x_k$$

The problem is to determine if there is a sequence of one or more integers $i_1, i_2, \dots, i_m$ such that:

$$\underline{w_1} \underline{w_{i_1} w_{i_2} \dots w_{i_m}} = \underline{x_1} x_{i_1} x_{i_2} \dots x_{i_m}$$

(w_i, x_i) is called a corresponding pair.

- Pair (w_1, x_1) is forced to be at the beginning of the two strings.

EXAMPLE

	A	B
i	w_i	x_i
1	<u>11</u>	1
2	<u>1</u>	111
3	<u>0111</u>	10
4	<u>10</u>	0

This MPCP instance has a solution: 3, 2, 2, 4:

$$\underline{w_1} w_3 w_2 w_2 w_4 = \underline{x_1} x_3 x_2 x_2 x_4 = \underline{1101111110}$$

EXAMPLE

	A	B
i	w_i	x_i
1	10	101
2	011	11
3	101	011

Does this MPCP instance have a solution?

MAPPING L_U TO MPCP

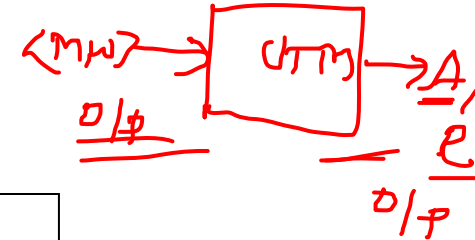
L_U instance

Given:
 (M, w)

Construct an
MPCP instance

MPCP instance

Two lists:
A and B



If M accepts w , the two lists can be matched.
Otherwise, the two lists cannot be matched.

MAPPING L_U TO MPCP

- Given M and w , there are five types of strings in list A and B:
- **Starting string (first pair):**

List A

List B

#

#q₀w#

where q_0 is the starting state of M .

$$L_U = \{ \langle n, w \rangle / \}$$

MAPPING L_U TO MPCP

$\Gamma = \{\underline{L}\}$

- Strings from the transition function δ :

List A

List B

qX

Yp

from $\delta(q, X) = (\underline{p}, Y, R)$ ←

ZqX

pZY

from $\delta(q, X) = (p, Y, \underline{L})$ ←

q#

Yp#

from $\delta(q, \underline{\#}) = (p, Y, R)$ ←

Zq#

pZY#

from $\delta(q, \underline{\#}) = (p, Y, \underline{L})$ ←

where Z is any tape symbol except the blank.

MAPPING L_U TO MPCP

- **Strings for copying:**

List A

X

List B

X

where X is any tape symbol (including the blank).

MAPPING L_U TO MPCP

- Strings for consuming the tape symbols at the end:

List A	List B
--------	--------

X <u>q</u>	q
------------	---

<u>q</u> Y	q
------------	---

<u>XqY</u>	q
------------	---

where q is an accepting state, and each X and Y is any tape symbol except the blank.

MAPPING L_U TO MPCP

- Ending string:

List A

q##

List B

#

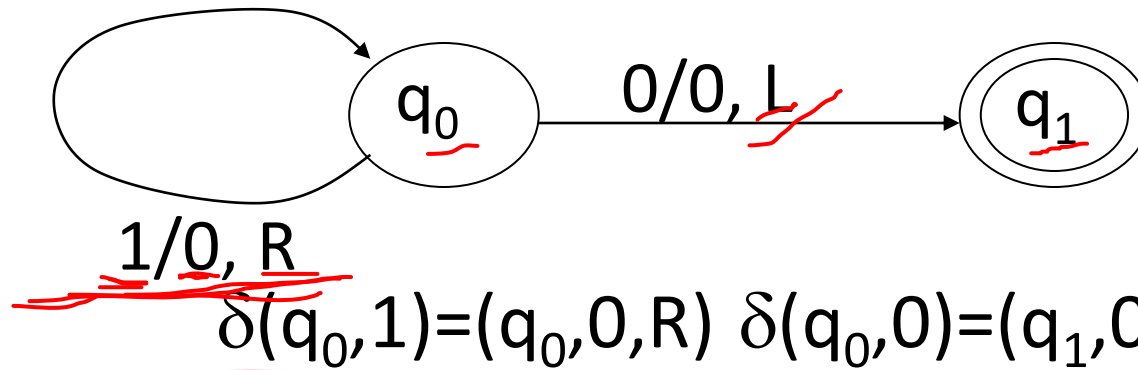
where q is an accepting state.

- Using this mapping, we can prove that the original L_U instance has a solution if and only if the mapped MPCP instance has a solution.

EXAMPLE

- Consider the following Turing machine:

$$M = (\{q_0, q_1\}, \{0, 1\}, \{0, 1, \#\}, \delta, q_0, \#, \{q_1\})$$



- Consider input $w=110$.

$q_0 \mid 110 \vdash 0q_010 \vdash 00q_00 \vdash 00q_10$

EXAMPLE

- Now we will construct an MPCP instance from M and w . There are five types of strings in list A and B:

A B
~~#~~ ~~#~~ q_0 w ~~#~~

- Starting string (first pair):

List A

#

List B

q_0 110#

EXAMPLE

- Strings from the transition function δ :

List A

q_01

$z=0$ $0q_00$

$z=1$ $1q_00$

List B

$0q_0$ (from $\delta(\overset{q}{q}_0, \overset{x}{1}) = (\overset{p}{q}_0, \overset{y}{0}, \underline{R})$) $\leftarrow$

q_100 (from $\delta(\underline{q}_0, 0) = (\underline{q}_1, 0, \underline{L})$) $\leftarrow$

q_110 (from $\delta(q_0, 0) = (q_1, 0, \underline{L})$)

$\Gamma = \{ \dots \# \}$

$$\delta(\overset{q}{q}, \overset{x}{x}) = (\overset{p}{p}, \overset{y}{y}, \underline{R})$$

$$\delta(\overset{q}{q}, \overset{x}{x}) = (\overset{p}{p}, \overset{y}{y}, \underline{L})$$

$z=qx \quad pz y$

$\underline{I}$

$\underline{F}$

EXAMPLE

- Strings for copying:

List A

0
1

List B

0
1

$\Gamma = \{\#, 0, 1\}$

EXAMPLE

- Strings for consuming the tape symbols at the end:

List A	List B	List A	List B
$x=0$ $x=1$ $y=0$ $y=1$ 0q ₁	<u>q₁</u>	0 <u>q₁</u> 1	q ₁
1q ₁	q ₁	1q ₁ 0	q ₁
<u>q₁0</u>	q ₁	0q ₁ 0	q ₁
<u>q₁1</u>	q ₁	1q ₁ 0	q ₁

$$\Gamma = \{0, 1, \textcircled{\#}, \underset{\times}{\textcircled{\#}}\}$$

$$\begin{array}{c} A \\ - \frac{xq}{qy} \\ \frac{xqy}{qy} \\ qy \end{array} \quad \begin{array}{c} B \\ q \\ q \\ q \end{array}$$

$$\frac{0, 1}{\underline{\quad}} \quad \begin{array}{c} 0 \quad 0 \\ 0 \quad 1 \\ 1 \quad 0 \\ 1 \quad 1 \end{array}$$

EXAMPLE

- Ending string:

List A

→ q₁##

List B

#

Now, we have constructed an MPCP instance.

EXAMPLE

<u>List A</u>	<u>List B</u>
1. <u>#</u>	<u>#q₀110#</u>
2. <u>q₀1</u>	<u>0q₀</u>
3. 0q ₀ 0	q ₁ 00
4. 1q ₀ 0	q ₁ 10
5. #	#
6. 0	0
7. 1	1
8. <u>q₁##</u>	<u>#</u>

<u>List A</u>	<u>List B</u>
9. 0q ₁	q ₁
10. 1q ₁	q ₁
11. q ₁ 0	q ₁
12. q ₁ 1	q ₁
13. 0q ₁ 1	q ₁
14. 1q ₁ 0	q ₁
15. 0q ₁ 0	q ₁
16. 1q ₁ 0\	q ₁

EXAMPLE OF ULP TO MPCP

- This ULP instance has a solution:

$q_0 1 1 0 \xrightarrow{\#} 0 q_0 1 0 \rightarrow 0 0 q_0 0 \rightarrow 0 q_1 0 0$ (halt)

- Does this MPCP instance has a solution?

List A:

$\# q_0 1 1 0 \# 0 q_0 1 0 \# 0 0 q_0 0 \# 0 q_1 0 0 \# q_1 0 \# q_1 \# \#$

List B:

$\# q_0 1 1 0 \# 0 q_0 1 0 \# 0 0 q_0 0 \# 0 q_1 0 0 \# q_1 0 \# q_1 \# \#$

The solution is the sequence of indices:

2, 7, 6, 5, 6, 2, 6, 5, 6, 3, 5, 15, 6, 5, 11, 5, 8

CLASS DISCUSSION

Consider the input $w = 101$. Construct the corresponding MPCP instance I and show that M will accept w by giving a solution to I .

CLASS DISCUSSION (CONT'D)

<u>List A</u>	<u>List B</u>
1. #	#q ₀ 101#
2. q ₀ 1	0q ₀
3. 0q ₀ 0	q ₁ 00
4. 1q ₀ 0	q ₁ 10
5. #	#
6. 0	0
7. 1	1
8. q ₁ ##	#

<u>List A</u>	<u>List B</u>
9. 0q ₁	q ₁
10. 1q ₁	q ₁
11. q ₁ 0	q ₁
12. q ₁ 1	q ₁
13. 0q ₁ 1	q ₁
14. 1q ₁ 0	q ₁
15. 0q ₁ 0	q ₁
16. 1q ₁ 0	q ₁

TEST YOUR KNOWLEDGE

- Which of the following problems is undecidable?
 - Deciding if a given context-free grammar is ambiguous.
 - Deciding if a given string is generated by a given context-free grammar.
 - Deciding if the language generated by a given context-free grammar is empty.
 - Deciding if the language generated by a given context-free grammar is finite.

SUMMARY

- What is undecidability
- Recursive and Recursive enumerable languages

REFERENCE

- Hopcroft J.E., Motwani R. and Ullman J.D, “Introduction to Automata Theory, Languages and Computations”, Second Edition, Pearson Education, 2008